

Teste Prático

Desenvolvedor Laravel

1. Cenário

Cenário: Sua equipe está desenvolvendo um **sistema integrado** onde são centralizadas todas as informações referentes aos projetos da organização.

O objetivo deste teste é implementar um módulo de **controle de estoque** com entrada e saída de materiais, em que todas as operações sejam devidamente registradas em um banco de dados relacional. O sistema deve suportar a **transferência de materiais entre diferentes almoxarifados**, sendo que cada almoxarifado possui nome e localização. A interface deve ser funcional e objetiva, permitindo consumir todas as funcionalidades disponíveis. Toda transação deve contar com **logging auditável**, e a solução como um todo deve prezar por observabilidade e boas práticas de desenvolvimento.

2. Desafios Técnicos

2.1 Arquitetura Inicial

Seguindo os padrões do Laravel, a estrutura inicial da aplicação deve contemplar, no mínimo, os seguintes componentes:

- **Models** — Representam a estrutura das tabelas no banco de dados. Devem refletir com fidelidade o schema das tabelas, incluindo a definição explícita dos relacionamentos (*hasMany*, *belongsTo*, etc.) e dos campos em *\$fillable* ou *\$guarded*.
- **Views (Blade)** — Responsáveis pelo HTML da aplicação. Não devem conter lógica de negócio; as diretivas `@php` / `@endphp` são vedadas. Todas as variáveis devem ser injetadas exclusivamente pelo Controller.
- **Controllers** — Centralizam as regras de negócio, validações, cálculos e orquestração das respostas. Devem ser enxutos, delegando lógica complexa para classes de serviço (Services) quando necessário.

2.2 Banco de Dados

Utilize o **MariaDB** como banco de dados principal. O schema deve ser gerenciado exclusivamente por meio de **Migrations** do Laravel, garantindo rastreabilidade e reprodutibilidade do ambiente. As tabelas devem contemplar, no mínimo:

- Almoxarifados (id, nome, localização, timestamps)
- Materiais (id, código interno, descrição, quantidade total, timestamps)
- Estoque por Almoxarifado — tabela pivot relacionando materiais e almoxarifados com quantidade
- Movimentações / Log de Transações (tipo, quantidade, almoxarifado origem/destino, timestamps, usuário)

2.3 Frontend — Blade / React / Angular

A interface deve ser desenvolvida utilizando um dos seguintes stacks de frontend:

Tecnologia	Finalidade	Link
Bootstrap 5	Estilização e responsividade geral	getbootstrap.com

DataTables	Listagem interativa de almoxarifados e materiais	datatables.net
SweetAlert2	Confirmações visuais das operações realizadas	sweetalert2.github.io

Diferencial: Renderize os dados no DataTables utilizando Server-Side Processing, consumindo um endpoint JSON dedicado no Controller.

2.4 Funcionalidades

A aplicação deve implementar, no mínimo, as seguintes funcionalidades acessíveis via interface web:

1. Cadastrar um almoxarifado (nome + localização).
2. Listar almoxarifados com paginação e busca.
3. Editar e excluir almoxarifados.
4. Cadastrar um material (código interno + descrição + quantidade).
5. Transferir um material entre almoxarifados — parcial ou totalmente.
6. Excluir um material.

Regra de negócio: Almoxarifados que possuam materiais associados **não podem ser excluídos**. Todos os materiais devem ser transferidos para outro almoxarifado antes que a exclusão seja permitida. Exiba uma mensagem clara ao usuário nesses casos.

2.5 DevOps

A solução deve ser containerizada. São esperados os seguintes artefatos:

- **Dockerfile** para a aplicação Laravel (PHP-FPM + Nginx ou similar).
- **docker-compose.yml** para orquestração local (app + MariaDB + Redis, se necessário).

2.6 Observabilidade e Logging

Implemente **logging estruturado** em uma tabela dedicada no banco de dados, registrando no mínimo: tipo de operação, payload relevante, usuário responsável, IP de origem e timestamp.

★ Diferencial

Integre métricas de monitoramento e rastreamento distribuído utilizando **OpenTelemetry** — incluindo traces de requisições HTTP e spans para operações críticas (transferências, exclusões). Demonstre os logs e métricas no README.

2.7 Testes Automatizados

★ Diferencial

Implemente testes **unitários** e/ou de **integração** cobrindo os principais endpoints da aplicação. Utilize o PHPUnit integrado ao Laravel ou o Pest. Os testes devem rodar via `php artisan test`.

3. Critérios de Avaliação

Critério	Peso
----------	------

Qualidade e organização do código (SOLID, Clean Code, Documentação)	Alta
Aderência à arquitetura MVC do Laravel	Alta
Corretude das funcionalidades implementadas	Alta
Tratamento de exceções e validações	Alta
Configuração de Docker	Média
Logging auditável	Média
Testes automatizados (diferencial)	Bônus
OpenTelemetry / Observabilidade avançada (diferencial)	Bônus
Server-Side DataTables (diferencial)	Bônus

4. Entrega

A entrega deve ser realizada com os seguintes requisitos atendidos:

- Código hospedado em repositório **GitHub** (público ou compartilhado com o avaliador).
- **README.md** detalhado com instruções de instalação, configuração de variáveis de ambiente, execução via Docker e lista de comandos necessários (migrations, seeders, etc.).
- A aplicação deve ser acessível em **localhost:3000** (ou porta configurável via `.env`).
- Demonstração dos **logs e métricas** no README, caso a observabilidade avançada tenha sido implementada.
- Aplicação de princípios **SOLID** e **Clean Code** de forma consistente em todo o projeto.
- Tratamento adequado de **exceções** — evite expor stack traces ao usuário final em produção.

Em caso de dúvidas sobre o escopo do teste, documente suas decisões de projeto no README.md e prossiga com a implementação. A capacidade de tomar decisões técnicas fundamentadas também é avaliada.